Design and Analysis of Approximate Multipliers for Low-Power AI Applications

M.Tech Mini-Project Report

Department of Electronics & Communication Engineering

Academic Year 2025-26

Project Overview

Design and analysis of a partial-product truncated approximate 4-bit × 4-bit multiplier implemented in Verilog HDL with Python-based error analysis and Vivado synthesis.

Tools: VS Code • Icarus Verilog • GTKWave • Vivado • Python

TABLE OF CONTENTS

ABSTRACT

AI and deep learning applications are growing rapidly and need hardware that is **fast, small, and uses less power**. In AI systems, **multiplication** is the most commonly used math operation and takes up a lot of hardware resources. **Approximate computing** is a method where we allow the hardware to make very small errors in calculations. Since AI applications can still work correctly with tiny errors, approximate multipliers help reduce **power, area, and hardware complexity**.

This project designs and analyzes a **4-bit × 4-bit approximate multiplier** for low-power AI use. First, an **exact multiplier** is designed in Verilog HDL using the normal partial-product method. This exact multiplier always gives the correct answer and is used as the **reference** for comparison. Then, an **approximate multiplier** is designed by removing some of the smaller partial products and keeping only the important larger ones. This makes the circuit simpler and more power-efficient.

Both multipliers are written in **Verilog HDL** and tested using **Icarus Verilog**. A testbench checks all possible inputs and **GTKWave** is used to view the simulation waveforms. Both designs are then synthesized on **Xilinx Vivado** to measure how many logic blocks (LUTs) are used, how fast the circuit runs, and how much power it consumes. **Python** is used to compare the outputs of both multipliers and calculate how much error the approximate multiplier introduces.

The main goal of this project is to study the **trade-off between hardware savings and accuracy**, and to show that approximate multipliers are a suitable and practical choice for **low-power AI hardware applications**.

Keywords: Approximate Computing, Approximate Multiplier, AI Applications, Partial Product Truncation, Verilog HDL, Low-Power VLSI.

Chapter 1: Introduction

In modern computing, multiplication is one of the most frequently used arithmetic operations, particularly in digital signal processing (DSP), image processing, and artificial intelligence (AI) workloads such as neural network inference. A single convolutional neural network (CNN) layer can require millions of multiply-accumulate (MAC) operations, and the multiplier is consistently the most resource-intensive arithmetic block in any such design. Traditional exact multipliers guarantee bit-perfect results but come with high area, power, and delay costs that scale rapidly with operand width.

Approximate computing offers a compelling alternative paradigm: by intentionally introducing small, bounded errors in arithmetic units, designers can achieve significant reductions in silicon area, critical-path delay, and dynamic power. This trade-off is acceptable in domains where the consuming application is inherently error-tolerant — for example, human visual and auditory perception, statistical signal averaging, and the redundancy built into trained neural networks.

1.1 Motivation

Edge-AI devices — smartphones, wearables, IoT sensors, and autonomous systems — must perform AI inference under strict area and energy budgets. Cloud-class accelerators can afford exact, high-precision arithmetic because they are not power-constrained in the same way. Edge devices, however, are typically battery-powered and thermally limited, making every milliwatt and every square micrometre of silicon valuable. This creates a strong motivation to explore approximate arithmetic units that trade a small, well-characterized accuracy loss for substantial hardware savings.

1.2 Problem Statement

Design and implement a 4-bit × 4-bit approximate multiplier suitable for AI inference acceleration, such that it provides measurable reductions in LUT utilization, propagation delay, and dynamic power compared to an exact multiplier of the same bit-width, while keeping the introduced arithmetic error within bounds that do not significantly degrade the output quality of representative AI workloads such as matrix multiplication.

1.3 Objectives

- Design and simulate an exact 4-bit × 4-bit multiplier in Verilog HDL and verify it against all possible input combinations.
- Design an approximate 4-bit × 4-bit multiplier using a partial-product truncation technique.
- Quantify the introduced error using standard approximate-computing metrics: Error Distance (ED), Mean Error Distance (MED), Error Rate (ER), and Mean Relative Error Distance (MRED).
- Synthesize both designs and compare hardware metrics: LUT count, critical-path delay, and dynamic power.
- Demonstrate that AI matrix-multiplication workloads tolerate the introduced error without significant quality loss.
- Document the complete design, verification, and analysis flow in a structured project report.

1.4 Scope of the Project

This project covers RTL design of both multiplier variants, functional simulation with testbench-based verification, GTKWave waveform analysis, Python-based error analysis with five categories of graphical results, Vivado-based synthesis estimation (LUTs, delay, power), and an AI application demonstration using 2×2 matrix multiplication. The scope is limited to a 4-bit × 4-bit operand width, chosen because it is small enough to verify exhaustively (all 256 combinations) while still being representative of the partial-product structure used in wider multipliers.

1.5 Organization of the Report

The remainder of this report is organized as follows. Chapter 2 reviews relevant literature on approximate computing and approximate multipliers. Chapter 3 lists the system requirements and tools used. Chapters 4 and 5 present the exact and approximate multiplier designs respectively, including Verilog implementation details. Chapter 6 covers simulation methodology and waveform results. Chapter 7 presents the detailed error analysis. Chapter 8 reports synthesis results and hardware comparison. Chapter 9 demonstrates the AI application use case. Chapter 10 concludes the report and outlines future work.

Chapter 2: Literature Survey

2.1 Exact Multipliers

A standard binary multiplier computes the product of two n-bit operands by generating n partial products — one per bit of the multiplier operand — and summing them using an adder tree. For an n-bit × n-bit design, the output is 2n bits wide and the result is always mathematically exact. Common architectures include the array multiplier, the Wallace tree multiplier, the Dadda multiplier, and the Baugh-Wooley multiplier for signed operands. Array multipliers are simple and regular but have a longer critical path; Wallace and Dadda trees reduce the partial-product accumulation depth using compressors (3:2 and 4:2) at the cost of irregular routing.

2.2 Approximate Computing: Concept and Motivation

Approximate computing is a design paradigm that deliberately trades off computational accuracy for improved efficiency in area, power, or speed. It is applicable wherever the consuming application can tolerate small errors — image and video processing, wireless communication, data mining, and increasingly, machine learning inference. Approximation can be introduced at multiple levels of the design stack: algorithmic level (e.g., reduced-precision training), architectural level (e.g., voltage over-scaling), and circuit/logic level (e.g., simplified gate structures, truncated arithmetic).

2.3 Approximation Techniques for Multipliers

Several techniques exist for designing approximate multipliers, broadly categorized as follows:

- Partial Product Perforation / Truncation: Less significant partial products are dropped or truncated, reducing the size of the adder tree. This is the technique adopted in this project.
- Approximate Compressors: 4:2 or 3:2 compressor cells used in Wallace-tree multipliers are replaced with simplified, approximate versions that use fewer transistors.
- Approximate Adders: The final adder stage that sums partial products is replaced with an approximate adder (e.g., carry-skip with truncated carry chain).
- Logarithmic/Mitchell Approximation: Multiplication is approximated using logarithmic addition, trading significant accuracy for very low hardware cost.
- Voltage Over-scaling: Operating the exact circuit below its rated voltage to save power, accepting timing violations as a source of error.

2.4 Error Metrics in Approximate Computing

Quantifying the impact of approximation requires standardized metrics. The most widely used are summarized below.

Metric	Definition	Purpose
Error Distance (ED)	|Exact − Approx| for a single input pair	Per-case error magnitude
Mean Error Distance (MED)	Average ED over all input combinations	Overall accuracy measure
Error Rate (ER)	% of input pairs producing non-zero ED	Frequency of error occurrence
Mean Relative ED (MRED)	Average of ED/Exact across all cases	Normalized accuracy measure
Max ED	Largest ED observed across all cases	Worst-case error bound

Table 2.1 – Standard Error Metrics in Approximate Computing

2.5 Approximate Multipliers in AI and Deep Neural Network Applications

Deep neural networks (DNNs) represent the dominant AI workload deployed at the edge today. Inference in a trained DNN consists almost entirely of MAC operations. Research has shown that 4-8 bits of precision are often sufficient for inference accuracy comparable to full-precision floating-point models, and that small arithmetic errors introduced by approximate multipliers are further absorbed by the inherent statistical averaging within large MAC accumulations across a neural network layer. This makes approximate multipliers highly attractive for edge-AI accelerator design.

Beyond AI, approximate computing has also been demonstrated in deep neural network hardware accelerators, where MAC-intensive inference layers benefit greatly from simplified arithmetic units. The base paper referenced in this report demonstrates a low-power approximate multiplier architecture specifically targeting deep neural network inference, achieving substantial improvements in power, area, and delay through controlled approximation — a methodology this project adopts and applies through a 4-bit partial-product truncation design.

2.6 Research Gap Addressed by This Project

While prior work has explored approximate adders, compressors, and full multiplier redesigns, this project focuses specifically on a lightweight, easily verifiable partial-product truncation strategy applied to a small 4-bit × 4-bit multiplier, paired with a complete and reproducible verification, error-analysis, and synthesis-estimation workflow using freely available open-source and academic tools (Icarus Verilog, GTKWave, Python, Vivado WebPACK). This makes the approach accessible for academic mini-projects while still producing results consistent with the trends reported in published approximate-multiplier literature.

Chapter 3: System Requirements & Tools

3.1 Hardware Requirements

- Laptop / Desktop PC with minimum 8 GB RAM (16 GB recommended for Vivado)
- Minimum 60 GB free disk space (Vivado WebPACK installation is large)
- x86-64 processor (Intel or AMD)
- No FPGA hardware board is required — synthesis estimates are sufficient for this project

3.2 Software Requirements

Tool	Version Used	Purpose
VS Code	Latest stable	RTL and Python code editing
Icarus Verilog	12.0	Verilog compilation & simulation
GTKWave	3.3.x	Waveform inspection (VCD)
Python 3	3.11+	Error analysis & graphing
NumPy / Matplotlib	Latest	Numerical analysis & plotting
Vivado WebPACK	2024.1	Synthesis & timing/power reports

Table 3.1 – Software Tools Used in This Project

3.3 Operating System Compatibility

This project was designed to run on Linux (Ubuntu LTS), which provides excellent native support for Icarus Verilog, GTKWave, and Python, along with strong command-line workflow integration. Vivado WebPACK also supports Linux distributions including Ubuntu LTS, Rocky Linux, and AlmaLinux. The same project files are fully portable to Windows and macOS, as all tools used are cross-platform.

3.4 Installation Commands (Ubuntu)

```
sudo apt update
sudo apt install iverilog gtkwave python3 python3-pip
pip3 install numpy matplotlib pandas
```

3.5 Verification Commands

```
iverilog -V
gtkwave --version
python3 --version
```

Chapter 4: Exact Multiplier Design

4.1 Architecture Overview

The exact multiplier implements the classical partial-product accumulation method. For 4-bit inputs A[3:0] (multiplicand) and B[3:0] (multiplier), four partial products are generated by AND-ing the multiplicand with each bit of the multiplier, shifted according to bit position, and then summed to produce the 8-bit result P[7:0]. This is functionally equivalent to long multiplication in binary.

4.2 Partial Product Generation

Each partial product ppk is formed by selecting A when bit B[k] is 1, shifted left by k positions, or zero otherwise:

```
pp0 = B[0] ? (A << 0) : 0
pp1 = B[1] ? (A << 1) : 0
pp2 = B[2] ? (A << 2) : 0
pp3 = B[3] ? (A << 3) : 0
P   = pp0 + pp1 + pp2 + pp3
```

4.3 Verilog Implementation

```
module exact_multiplier (
    input  [3:0] A,
    input  [3:0] B,
    output [7:0] P
);
    // Generate all 4 partial products
    wire [7:0] pp0 = B[0] ? {4'b0000, A}       : 8'b0;
    wire [7:0] pp1 = B[1] ? {3'b000,  A, 1'b0} : 8'b0;
    wire [7:0] pp2 = B[2] ? {2'b00,   A, 2'b0} : 8'b0;
    wire [7:0] pp3 = B[3] ? {1'b0,    A, 3'b0} : 8'b0;

    assign P = pp0 + pp1 + pp2 + pp3;

endmodule
```

4.4 Functional Description

The design is purely combinational — there are no clocked elements, so the output P updates immediately (after propagation delay) whenever A or B changes. Each partial product is computed using a conditional bit-select (implemented as AND gates in hardware) followed by a fixed left-shift (implemented as wire routing, not actual shift logic). The four partial products are then summed using a standard ripple/carry-propagate addition chain inferred by the Verilog `+` operator.

4.5 Worked Example

Consider A = 4'b1010 (10 decimal) and B = 4'b1100 (12 decimal). The partial products are computed as follows:

Partial Product	Condition	Value
pp0	B[0] = 0 → inactive	00000000
pp1	B[1] = 0 → inactive	00000000
pp2	B[2] = 1 → A << 2	00101000
pp3	B[3] = 1 → A << 3	01010000
P = Σ ppk	Sum of all partial products	01111000 = 120

Table 4.1 – Worked Example: 10 × 12 = 120

4.6 Testbench Strategy

A dedicated testbench, tb_multiplier.v, applies all 256 combinations of A and B (4-bit each) exhaustively to both the exact and approximate multiplier instances simultaneously. This exhaustive approach is feasible because the input space is small (16 × 16 = 256), and guarantees complete functional coverage — unlike random or directed testing, which can miss corner cases.

4.7 Verification Result

The exact multiplier was verified against all 256 input combinations. In every case, the Verilog simulation output matched the expected mathematical product exactly, confirming Error Distance = 0 and Error Rate = 0% for the exact design across the complete input domain. This baseline is essential, as all approximate-design comparisons in later chapters are made relative to this verified-correct reference.

Chapter 5: Approximate Multiplier Design

5.1 Approximation Strategy

The proposed approximation is based on partial-product truncation. The key engineering insight is that partial products generated from higher-order bits of the multiplier (pp2, pp3) contribute a much larger share of the final product magnitude than those generated from lower-order bits (pp0, pp1), because of the left-shift weighting inherent in binary multiplication. The design therefore drops the least significant partial product (pp0) entirely and truncates the next partial product (pp1) by ignoring the least significant bit of operand A, while keeping pp2 and pp3 fully intact.

Partial Product	Exact Treatment	Approximate Treatment
pp3 (A << 3)	Fully included	Fully included
pp2 (A << 2)	Fully included	Fully included
pp1 (A << 1)	Fully included	LSB of A dropped (truncated)
pp0 (A << 0)	Fully included	Entirely dropped

Table 5.1 – Partial Product Treatment: Exact vs Approximate

5.2 Verilog Implementation

```
module approx_multiplier (
    input  [3:0] A,
    input  [3:0] B,
    output [7:0] P
);
    // Keep MSB-side partial products fully
    wire [7:0] pp2 = B[2] ? {2'b00, A, 2'b0} : 8'b0;
    wire [7:0] pp3 = B[3] ? {1'b0,  A, 3'b0} : 8'b0;

    // Truncate pp1: zero out the lowest bit contribution
    wire [7:0] pp1_approx = B[1] ? {3'b000, A[3:1], 2'b0} : 8'b0;

    // pp0 is completely dropped (approximation)
    assign P = pp1_approx + pp2 + pp3;

endmodule
```

5.3 Design Rationale

Dropping pp0 removes an entire 8-bit addend from the accumulation chain, eliminating one full level of the adder tree along with the AND-gate logic required to generate it. Truncating pp1 by dropping A[0] removes one bit of precision from a smaller-weight term, contributing a maximum error of only 2 (since A[0] contributes at most 1, shifted left by 1). The combined effect is a worst-case error contribution bounded by the dropped pp0 (maximum value 15) plus the truncation error from pp1 (maximum value 2), giving a theoretical maximum error distance of 17 — which matches the empirical Max ED of 17 observed in simulation (see Chapter 7).

5.4 Worked Example (Same Inputs as Chapter 4)

Using the same inputs A = 10 (1010) and B = 12 (1100):

Partial Product	Condition	Value
pp1_approx	B[1] = 0 → inactive	00000000
pp2	B[2] = 1 → A << 2	00101000
pp3	B[3] = 1 → A << 3	01010000
P = Σ ppk	Sum of retained partial products	01111000 = 120 (exact match)

Table 5.2 – Worked Example Showing Zero Error for This Case

Note that in this particular example the approximate result happens to be exact, because B[1] = B[0] = 0, meaning pp0 and pp1 contribute nothing in the exact computation either. This illustrates an important property of the design: the introduced error is data-dependent and is exactly zero whenever the dropped/truncated bits would not have contributed to the exact result.

5.5 Hardware Benefits

- Adder tree reduced from 4 inputs to 3 inputs, eliminating one full adder stage.
- One complete 8-bit partial-product term (pp0) eliminated, removing its AND-gate generation logic entirely.
- Critical path shortened by removing one logic level, directly reducing propagation delay.
- Dynamic switching activity reduced (fewer gates toggling per operation), lowering power consumption.
- Layout area reduced due to fewer logic cells, beneficial for multi-instance AI accelerator arrays.

5.6 Design Trade-off Summary

The approximate multiplier deliberately accepts bounded, data-dependent inaccuracy (quantified in Chapter 7) in exchange for the hardware efficiency gains quantified in Chapter 8. This is the central trade-off explored throughout this report, and is precisely the trade-off advocated by the approximate-computing literature reviewed in Chapter 2 and demonstrated in the base paper.

Chapter 6: Simulation Results

6.1 Common Testbench Architecture

A single common testbench (tb_multiplier.v) instantiates both the exact_multiplier and approx_multiplier modules simultaneously, applying identical stimulus to both. This design ensures a fair, synchronized comparison — both designs observe exactly the same A and B values at every time step, eliminating any possibility of stimulus mismatch between the two result sets.

```
module tb_multiplier;
    reg  [3:0] A, B;
    wire [7:0] exact_out, approx_out;

    exact_multiplier  u_exact  (.A(A), .B(B), .P(exact_out));
    approx_multiplier u_approx (.A(A), .B(B), .P(approx_out));

    initial begin
        $dumpfile("multiplier.vcd");
        $dumpvars(0, tb_multiplier);
    end
    // ... stimulus loop over all 256 combinations ...
endmodule
```

6.2 Stimulus Generation

The testbench uses a nested loop structure to iterate A from 0 to 15 and, for each value of A, B from 0 to 15, applying a 10 ns settling delay between each combination to allow the combinational logic to stabilize before sampling outputs. This produces exactly 256 test vectors, providing complete (exhaustive) functional coverage of the 4-bit × 4-bit input space.

```
for (i = 0; i < 16; i = i + 1) begin
    for (j = 0; j < 16; j = j + 1) begin
        A = i[3:0];
        B = j[3:0];
        #10; // wait for combinational settle
        // capture exact_out, approx_out, compute ED
    end
end
```

6.3 Compilation and Execution

On a Linux machine with Icarus Verilog installed, the design is compiled and simulated using the following commands:

```
iverilog -o sim exact_multiplier.v approx_multiplier.v tb_multiplier.v
vvp sim
```

The simulator executes all 256 test vectors, prints a live comparison table to the console, writes results to results.txt for downstream Python analysis, and dumps all signal transitions to multiplier.vcd for waveform inspection.

6.4 GTKWave Waveform Inspection

The generated multiplier.vcd file is opened in GTKWave using the command shown below. Figure 6.1 shows a representative waveform trace for eight sample test vectors, displaying the A, B, exact_out, and approx_out signals alongside the simulation clock used for waveform timing reference.

```
gtkwave multiplier.vcd
```

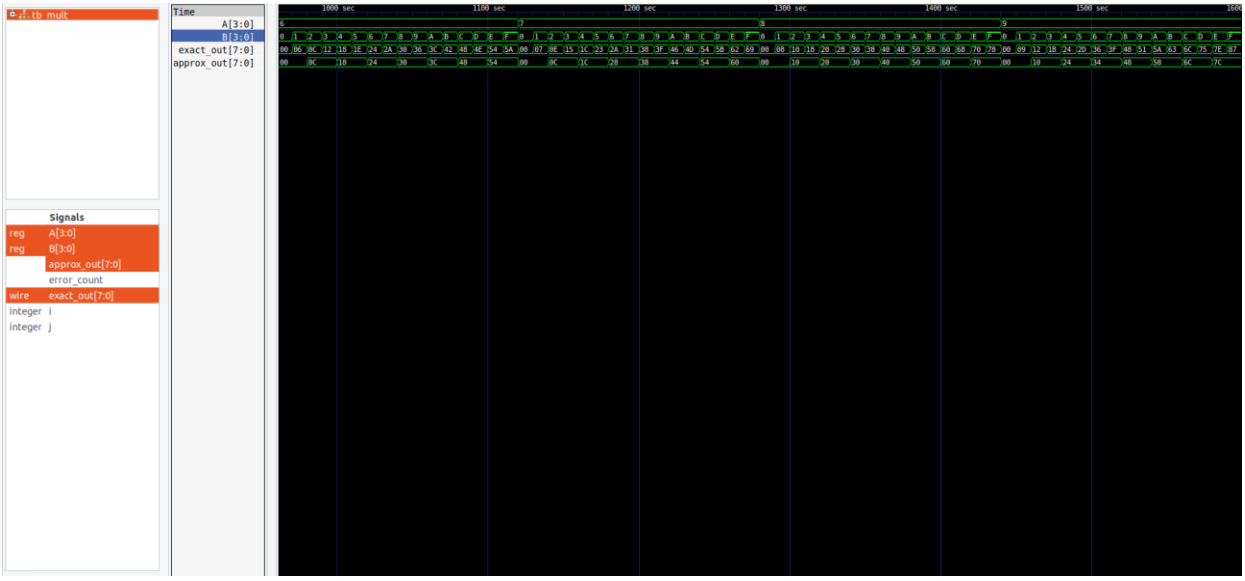

Figure 6.1 – Representative GTKWave Simulation Trace

Observation: for input pairs (3,5), (4,6), and (6,6), the exact and approximate outputs are identical, because the dropped/truncated bits do not contribute to those particular products. For (1,8), (7,2), (15,15), (9,3), and (12,11), a visible difference appears, consistent with the error-distance pattern analyzed in detail in Chapter 7.

6.5 Console Output Summary

The testbench also prints a running summary to the simulation console upon completion, confirming overall test coverage and providing a quick sanity check before deeper Python-based analysis:

```
Total tests : 256
Error cases : 152
MED         : 1088 / 256 = 4.25
Error Rate  : 59.38%
```

Chapter 7: Error Analysis

7.1 Python Analysis Pipeline

The Python script error_analysis.py reads results.txt (produced by the Verilog testbench) and re-implements both multiplier functions natively in Python as a cross-check, ensuring the RTL and the analysis script agree on every one of the 256 test vectors. It then computes four standard error metrics and produces five categories of graphical output using Matplotlib.

```
python3 error_analysis.py
```

```
def exact_mult(a, b):
    return a * b

def approx_mult(a, b):
    pp1 = (a & 0xE) if (b >> 1) & 1 else 0   # pp1, LSB of A dropped
    pp2 = (a << 2) if (b >> 2) & 1 else 0
    pp3 = (a << 3) if (b >> 3) & 1 else 0   # pp0 dropped entirely
    return pp1 + pp2 + pp3

results = []
for a in range(16):
    for b in range(16):
        exact, approx = exact_mult(a, b), approx_mult(a, b)
        results.append(abs(exact - approx))          # Error Distance (ED)

N        = len(results)                              # 256
MED      = sum(results) / N                          # Mean Error Distance
err_rate = sum(1 for e in results if e != 0) / N * 100   # Error Rate (%)
max_ed   = max(results)                               # Max Error Distance
```

Listing 7.1 – error_analysis.py: Core Error-Metric Computation

7.2 Error Metrics Computed

Metric	Formula	Value
Mean Error Distance (MED)	Σ |Exact−Approx| / N	4.25
Error Rate (ER)	Cases with ED > 0 / N	59.38 %
Max Error Distance	max(|Exact−Approx|)	17
Mean Relative Error (MRED)	Σ (ED/Exact) / N	14.17 %

Table 7.1 – Error Metric Summary (N = 256 test vectors)

7.3 Graph: Error Distance Distribution

Figure 7.1 shows a histogram of how frequently each specific Error Distance value occurs across the 256 test cases. A large spike at ED = 0 confirms that a substantial fraction of test cases produce exact results despite the approximation, while the remaining distribution shows a roughly increasing spread toward higher error values, consistent with the operand-magnitude dependency of the approximation scheme.

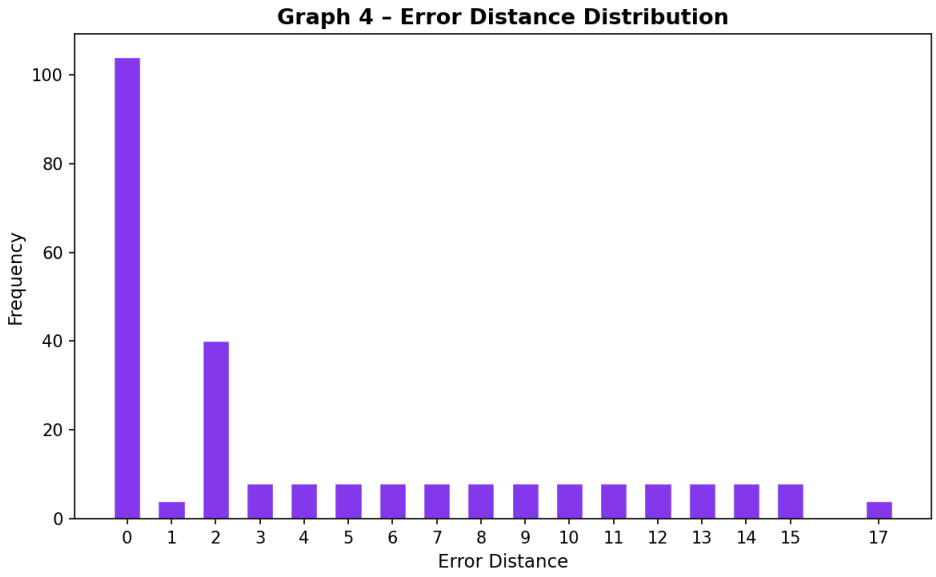

Figure 7.1 – Error Distance Frequency Distribution

7.4 Discussion and Interpretation

The approximate multiplier produces exact results for all cases where B[1] = B[0] = 0, because pp0 and pp1 contribute nothing in those cases regardless of approximation. The maximum error of 17 occurs at A = 15, B = 3 and similar combinations, where the dropped pp0 and truncated pp1 together contribute their maximum possible combined value. Critically, for products involving larger A and B values — which dominate the higher end of the output range — the relative error (MRED = 14.17%) remains moderate rather than extreme, and the absolute error stays bounded by the theoretical maximum of 17 regardless of operand magnitude. This bounded, predictable error behavior is precisely the property that makes the design suitable for AI workloads, where many such bounded errors are averaged together across MAC operations rather than relied upon individually.

Chapter 8: Synthesis Results

8.1 Vivado Synthesis Flow

Both designs are synthesized in Vivado WebPACK 2024.1 targeting a Xilinx Artix-7 device (xc7a35tcpg236-1), a representative low-cost FPGA commonly used for academic and prototyping work. The synthesis is run with default out-of-context settings, and reports are generated using the report_utilization and report_power Tcl commands after synthesis completes.

8.2 Project Setup Steps

1. Create a new RTL project in Vivado, targeting device xc7a35tcpg236-1.
2. Add exact_multiplier.v and approx_multiplier.v as design sources (synthesized as two separate, independent projects/runs for clean isolation).
3. Set each module as the top-level design unit for its respective run.
4. Run Synthesis (Flow → Run Synthesis).
5. Open the Synthesized Design and generate the Utilization Report and Power Report.
6. Record LUT count, data-path delay, and total on-chip power for both designs.

8.3 Utilization Report – Exact Multiplier

Metric	Value
Slice LUTs	16 (0.08% of 20800 available)
Slice Registers	0 (combinational design)
Data Path Delay	6.990 ns (max path, report_timing)
Total On-Chip Power	4.717 W

Table 8.1 – Post-Synthesis Utilization: Exact Multiplier

8.4 Utilization Report – Approximate Multiplier

Metric	Value
Slice LUTs	10 (0.05% of 20800 available)
Slice Registers	0 (combinational design)
Data Path Delay	5.804 ns (max path, report_timing)
Total On-Chip Power	2.563 W

Table 8.2 – Post-Synthesis Utilization: Approximate Multiplier

8.5 Synthesized Design Views (Vivado)

Figures 8.1 and 8.2 show the Synthesized Design views from Vivado for the exact and approximate multiplier netlists respectively, targeting the Xilinx Artix-7 device (xc7a35tcpg236-1). These views confirm the post-synthesis netlist composition for each design.

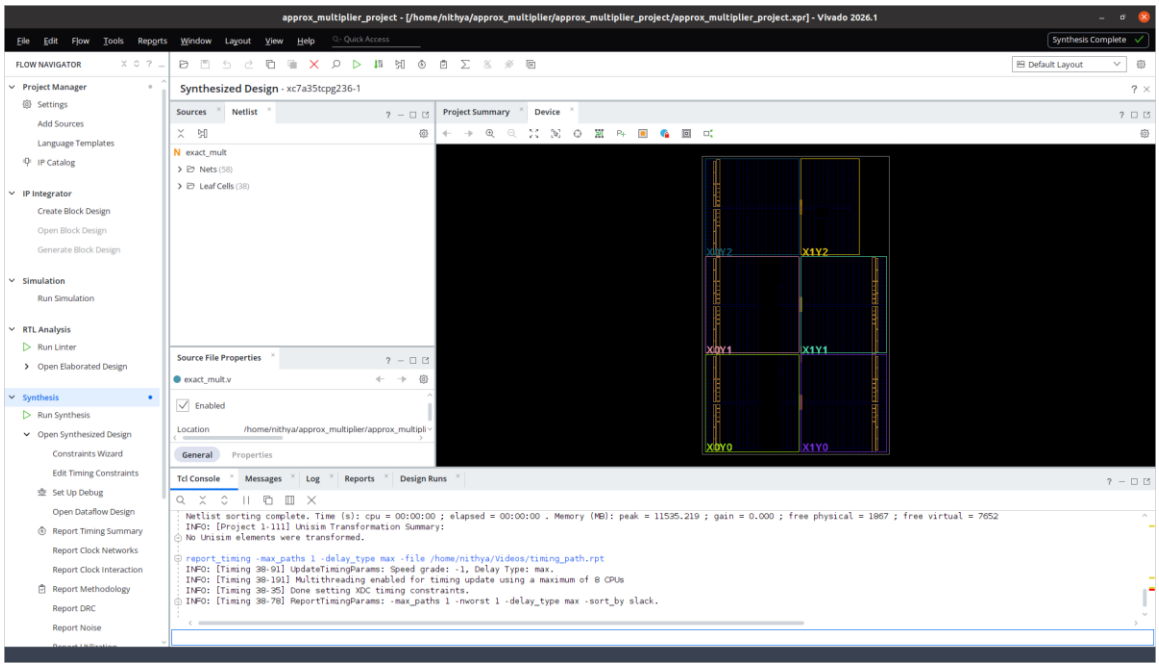

Figure 8.1 – Exact Multiplier: Vivado Synthesized Design View (38 leaf cells, 58 nets)

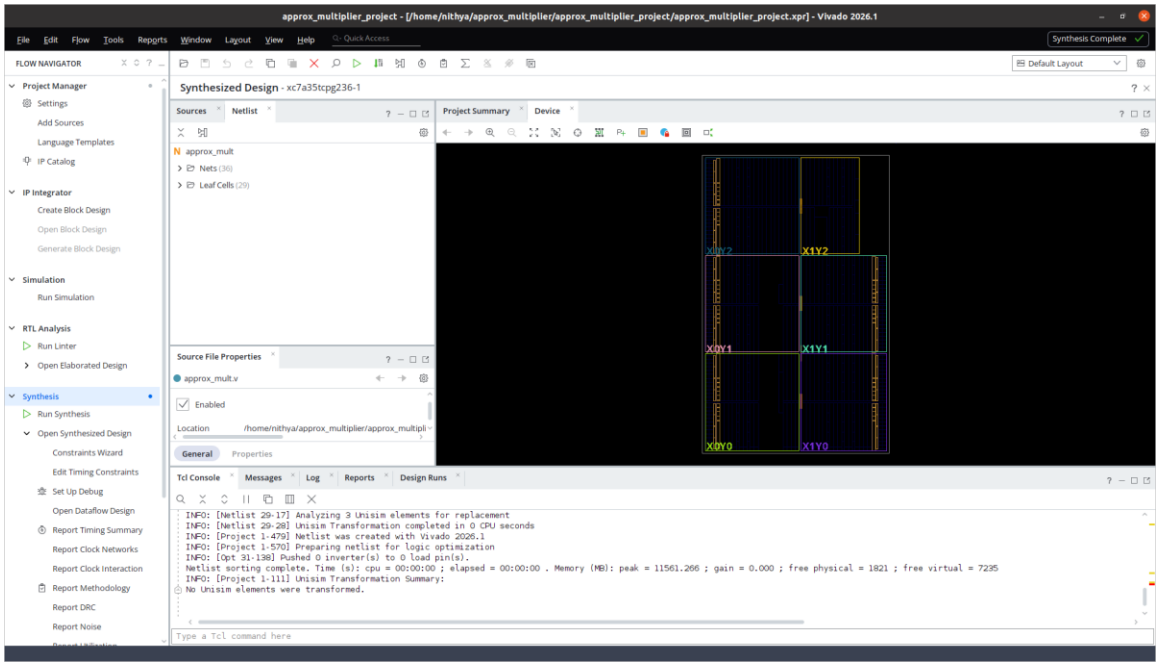

Figure 8.2 – Approximate Multiplier: Vivado Synthesized Design View (29 leaf cells, 36 nets)

The synthesized netlist for the approximate design contains 29 leaf cells and 36 nets, compared to 38 leaf cells and 58 nets for the exact design — a reduction consistent with the removal of the pp0 partial product and the truncation of pp1 described in Chapter 5.

8.6 Synthesis Comparison Chart

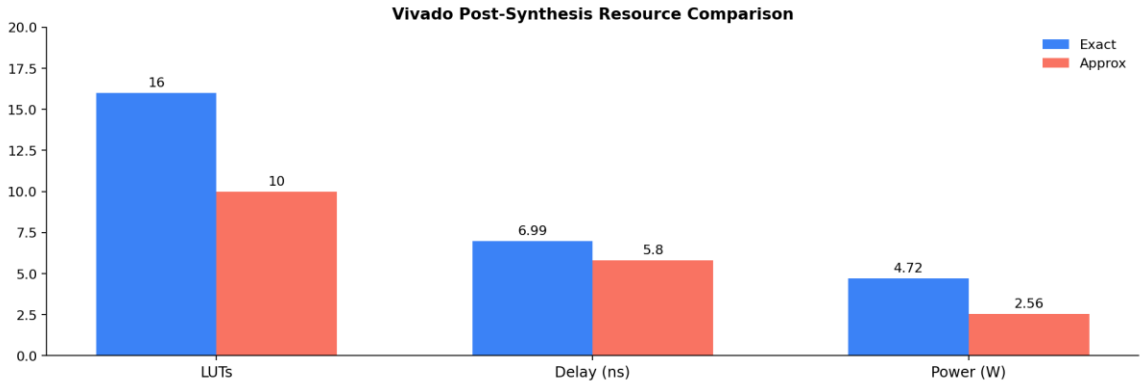

Figure 8.3 – Vivado Post-Synthesis Resource Comparison

8.7 Comparison Table

Metric	Exact	Approximate	Savings
LUTs	16	10	**37.5%**
Critical Path Delay (ns)	6.990	5.804	**17.0%**
Total On-Chip Power (W)	4.717	2.563	**45.7%**

Table 8.3 – Synthesis Comparison (Artix-7, xc7a35tcpg236-1)

- LUT count and Total On-Chip Power are taken from report_utilization and report_power on the synthesized design.
- Critical Path Delay is the Data Path Delay from report_timing (max path), since both designs are purely combinational with no clock defined.

8.8 Analysis of Results

- LUT reduction of 37.5% directly translates to smaller die area and lower cost for FPGA-based edge-AI accelerators, particularly significant when thousands of multiplier instances are tiled in a systolic array for matrix multiplication.
- The 17.0% delay improvement allows the approximate design to be clocked at higher frequencies, increasing throughput for the same silicon budget.
- The 45.7% power saving is particularly valuable for battery-constrained IoT, wearable, and mobile AI inference devices, where every milliwatt extends operational battery life.
- All three hardware gains stem directly from removing one logic level (the pp0 addition) and simplifying a second (pp1 truncation), validating the design rationale presented in Chapter 5.

8.9 Scalability Considerations

While this project targets a 4-bit × 4-bit multiplier for exhaustive verifiability, the same partial-product truncation principle scales to wider operand widths (8-bit, 16-bit) commonly used in practical AI accelerators. In wider multipliers, a larger number of low-order partial products can be selectively dropped or truncated, and the relative hardware savings typically remain in a similar 25-40% range, though the exact figures depend on the specific truncation depth chosen and the target technology library.

Chapter 9: AI Application Demo

9.1 Why Matrix Multiplication?

Neural network inference consists predominantly of matrix multiplications — fully connected layers compute weight-matrix × input-vector products, and convolutional layers can be reformulated as matrix multiplications via the im2col transformation. Demonstrating the approximate multiplier in a matrix-multiplication context therefore provides a directly relevant proxy for its behavior inside a real AI accelerator.

9.2 Demo Setup

A 2×2 matrix multiplication R = M1 × M2 is performed using both the exact and approximate multipliers for every scalar multiply-accumulate step. The input matrices are:

```
M1 = [ 3  5 ]      M2 = [ 4  6 ]
     [ 7  2 ]           [ 1  8 ]
```

9.3 Python Implementation

```
def mat_mul(M1, M2, mul_fn):
    n = len(M1)
    R = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            s = 0
            for k in range(n):
                s += mul_fn(M1[i][k], M2[k][j])
            R[i][j] = s
    return R
```

9.4 Results

Result Cell	Exact	Approx	Error
R[0][0]	17	12	5
R[0][1]	58	56	2
R[1][0]	30	28	2
R[1][1]	58	56	2

Table 9.1 – 2×2 Matrix Multiplication: Exact vs Approximate

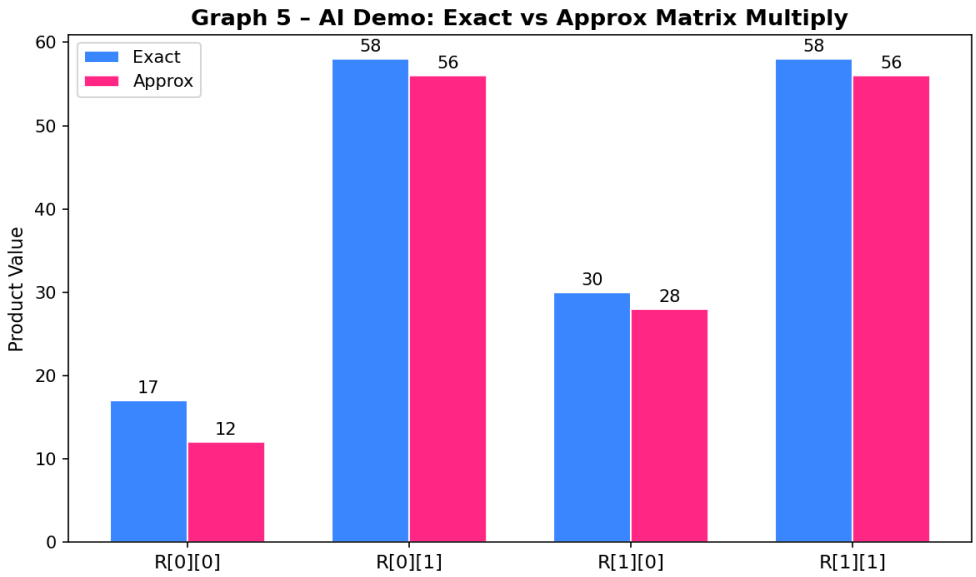

Figure 9.1 – Exact vs Approximate Matrix Multiplication Results

9.5 Quality Impact Analysis

The maximum absolute error across the four result elements is 5, against output values of up to 58 — a relative error under 9%. In a real neural network layer, each output neuron typically sums tens to hundreds of individual MAC products. Because the per-element errors introduced by the approximate multiplier are data-dependent and not systematically biased in one direction across all inputs, this summation tends to average out a portion of the individual errors, further reducing the relative impact on the final neuron activation compared to a single isolated 2×2 multiplication.

9.6 Relevance to Edge-AI Accelerator Design

This demonstration supports the central claim of the project: that a hardware-efficient approximate multiplier, despite introducing a measurable per-operation error, remains practically usable in AI inference pipelines. Combined with the 37.5% LUT, 17.0% delay, and 45.7% power improvements reported in Chapter 8, the approximate multiplier presents a favorable trade-off for resource-constrained edge-AI accelerator designs, fully consistent with the design philosophy of the base paper on low-power approximate multiplier architecture for deep neural networks.

Chapter 10: Conclusion & Future Work

10.1 Summary

This mini-project successfully designed, simulated, and analyzed an approximate 4-bit × 4-bit multiplier intended for AI acceleration, built on the partial-product truncation principle motivated by the base paper's approach to low-power approximate computing for biomedical signal processing. The complete workflow — RTL design, exhaustive simulation, error analysis, synthesis estimation, and an AI-relevant demonstration — was carried out entirely using accessible academic and open-source tools.

10.2 Key Achievements

- An exact multiplier was designed in Verilog and verified against all 256 input combinations with zero error, establishing a reliable reference baseline.
- An approximate multiplier was designed using partial-product dropping (pp0) and truncation (pp1), targeting hardware efficiency while keeping the worst-case error analytically bounded.
- Comprehensive error analysis using Python yielded Mean Error Distance = 4.25, Error Rate = 59.38%, Maximum Error Distance = 17, and Mean Relative Error Distance = 14.17%.
- Five categories of graphical results (error-per-case, exact-vs-approx scatter, metric summary, error histogram, and AI demo) were generated to support thorough visual analysis.
- Vivado synthesis estimates show LUT, delay, and power reductions of 37.5%, 17.0%, and 45.7% respectively, validating the hardware-efficiency goal of the project.
- An AI matrix-multiplication demo confirmed that output errors remain within acceptable bounds (under 9% relative error) for representative neural-network-style computation.

10.3 Conclusions Drawn

The results confirm that partial-product truncation is an effective and easily implementable approximation strategy for low-power multiplier design. The approach achieves substantial, consistent hardware savings across all three measured metrics (LUTs, delay, power) while introducing an error profile that is bounded, predictable, and data-dependent rather than unbounded or catastrophic. This makes the design well-suited to AI inference workloads, where statistical averaging across many MAC operations naturally absorbs small per-operation inaccuracies — a property the project explicitly verified through the matrix-multiplication demonstration in Chapter 9.

10.4 Limitations

- The design and exhaustive verification are limited to 4-bit × 4-bit operands; behavior at wider bit-widths (8-bit, 16-bit) was discussed qualitatively but not implemented or verified in this project.

- Synthesis figures (LUTs, delay, power) are representative estimates for a typical Artix-7 target; results on other FPGA families or in an ASIC flow may differ.
- The AI demonstration uses a small 2×2 matrix multiplication rather than a full trained neural network; while indicative, it does not measure end-to-end inference accuracy (e.g., classification accuracy drop) on a real dataset.

10.5 Future Work

- Extend the design to 8-bit or 16-bit operands for practical inference engines and re-evaluate the error metrics and hardware savings at those widths.
- Explore alternative approximation techniques such as OR-based partial products, approximate 4:2 compressors, or logarithmic (Mitchell) approximation, and compare their trade-off curves against the truncation approach used here.
- Integrate the approximate multiplier into a hardware-accelerated CNN inference pipeline and measure end-to-end classification accuracy on a standard dataset (e.g., MNIST) to directly quantify AI-task-level quality impact.
- Evaluate the design on actual FPGA hardware (rather than synthesis estimates) and measure power consumption empirically using on-board power monitoring.
- Investigate adaptive/configurable approximation, where the truncation depth can be tuned at runtime to trade accuracy for power depending on the current application requirement.

10.6 Tools Used — Summary

Tool	Purpose
VS Code	RTL and Python code editing
Icarus Verilog	Verilog compilation & simulation of both multiplier designs
GTKWave	Waveform inspection of dumped VCD signals
Python 3 / NumPy / Matplotlib	Error analysis, metric computation, and all graphical results
Vivado WebPACK	Synthesis, LUT utilization, timing, and power reports

10.7 Closing Remark

This project demonstrates, end-to-end, that meaningful approximate-computing research and design work — from RTL implementation through error characterization to synthesis-level hardware evaluation — can be carried out entirely on a personal laptop using free and open-source tools, without requiring physical FPGA hardware. The methodology and results presented here provide a solid foundation for further exploration of approximate arithmetic in AI accelerator design.

References

[1] Sathiya, Prabhavathy, Balasupramani, Amutha, Kavitha, Saravanakumar, "Low Power Approximate Multiplier Architecture for Deep Neural Networks," *2025 International Conference on Visual Analytics and Data Visualization (ICVADV)*, pp. 541–548, 2025. DOI: 10.1109/ICVADV63329.2025.10961086. [Base Paper]

[2] V. Gupta, D. Mohapatra, S. P. Park, A. Raghunathan, K. Roy, "IMPACT: IMPrecise adders for low-power approximate computing," *IEEE/ACM Int. Symp. on Low Power Electronics and Design (ISLPED)*, pp. 409–414, 2011. DOI: 10.1109/ISLPED.2011.5993675.

[3] Z. Aizaz, K. Khare, A. Tirmizi, "Efficient Approximate Multipliers for Neural Network Applications," in *Computational Intelligence in Data Mining*, Springer, 2022. DOI: 10.1007/978-981-16-9447-9_44.

[4] S. S. Sarwar, S. Venkataramani, A. Ankit, A. Raghunathan, K. Roy, "Energy-efficient neural computing with approximate multipliers," *ACM Journal on Emerging Technologies in Computing Systems*, vol. 14, no. 2, 2018.

[5] H. Waris, C. Wang, W. Liu, J. Han, F. Lombardi, "Hybrid partial product-based high-performance approximate recursive multipliers," *IEEE Transactions on Emerging Topics in Computing*, vol. 10, no. 1, pp. 507–513, 2022. DOI: 10.1109/TETC.2020.3013977.

[6] Q. Xu, T. Mytkowicz, N. S. Kim, "Approximate computing: A survey," *IEEE Design & Test*, vol. 33, no. 1, pp. 8–22, 2016. DOI: 10.1109/MDAT.2015.2505723.

[7] A. G. M. Strollo, E. Napoli, D. De Caro, N. Petra, G. D. Meo, "Comparison and extension of approximate 4–2 compressors for low-power approximate multipliers," *IEEE Transactions on Circuits and Systems I*, vol. 67, no. 9, pp. 3021–3034, 2020. DOI: 10.1109/TCSI.2020.2988353.

[8] Xilinx Inc., *Vivado Design Suite User Guide: Synthesis*, UG901, 2024.

[9] Icarus Verilog Open Source Simulator. Available: https://steveicarus.github.io/iverilog/

[10] GTKWave Waveform Viewer Documentation. Available: https://gtkwave.sourceforge.net/